# Persistencia de datos en el navegador

| | |
|---|---|
| **Universidad** | Universidad de Caldas |
| **Asignatura** | Programación Web Creativa II |
| **Docente** | Andrés Felipe Osorio Quintero |
| **Perfil docente** | Ingeniero de Software, desarrollador web y marketing digital |
| **Unidad** | 2. Interactividad avanzada y almacenamiento local |
| **Subtema** | Persistencia de datos en el navegador |
| **Documento fuente** | 3-Persistencia de datos en el navegador.pdf |
| **Código base** | Codigo HTML CSS JS.docx (lista de tareas) |

| **Estudiante:** | **Código:** | **Fecha:** |
|---|---|---|
| Juan Manuel Orozco<br>Mariana Díaz<br>Eliseth Nathaly | | |

**Indicaciones:** responde las preguntas 1 a 5 con tus propias palabras. Desarrolla los ejercicios 6 a 20 en Visual Studio Code y adjunta las evidencias solicitadas debajo de cada uno.

## PARTE I. TEORÍA

*Preguntas 1 a 5 · Respuesta abierta y desarrollada*

### 1. Pérdida de estado y mecanismos de almacenamiento

Explica por qué una aplicación web pierde su estado al recargar la página y cuáles son las dos grandes estrategias para resolverlo. Ubica a Web Storage dentro de los mecanismos de almacenamiento del navegador (cookies, IndexedDB y Cache API) y justifica en qué casos es la herramienta adecuada.

**Por qué se pierde el estado.** Una página web es un documento cuyo estado (variables de JavaScript, contenido del DOM, lo que el usuario escribió o eligió) vive solo en la memoria del navegador mientras ese documento está cargado. Al recargar, el navegador descarta el documento, lo vuelve a pedir al servidor y ejecuta el código desde cero: las variables se reinician y el DOM se reconstruye. Además, HTTP es un protocolo sin estado, así que entre una petición y otra nada se conserva por sí solo.

**Dos grandes estrategias.** (1) Persistir el estado en el cliente, es decir, en el propio navegador (Web Storage, cookies, IndexedDB o Cache API). (2) Persistirlo en el servidor (base de datos, sesiones o una API) y recuperarlo en cada visita. En la práctica se combinan: el servidor guarda lo importante y el navegador guarda lo cómodo o temporal.

**Dónde queda Web Storage.** Web Storage (localStorage y sessionStorage) es el almacén clave-valor más simple del navegador: es síncrono, solo guarda texto, ofrece cerca de 5 MB por origen y no se envía al servidor. Las cookies son más pequeñas (alrededor de 4 KB) y viajan con cada petición HTTP, por lo que sirven para que el servidor reconozca al usuario. IndexedDB es una base de datos asíncrona y transaccional para grandes volúmenes de datos estructurados (objetos, archivos, índices). La Cache API guarda pares petición/respuesta HTTP y se usa sobre todo con Service Workers para tener

recursos sin conexión.

**Cuándo es la herramienta adecuada.** Cuando los datos son pequeños, simples y no sensibles: preferencias de la interfaz (tema, idioma), borradores de formularios, el estado de una lista o una caché ligera con vigencia. No es adecuada para datos grandes o muy estructurados (IndexedDB), para credenciales o información sensible, ni para datos de los que dependa la seguridad o la lógica de negocio (servidor).

## 2. localStorage, sessionStorage y cookies

Compara localStorage, sessionStorage y las cookies en cuanto a duración, alcance, capacidad y envío al servidor. Argumenta qué mecanismo elegirías para guardar el tema visual, el borrador de un formulario y el identificador de una sesión autenticada.

**Comparación.** Las tres guardan pares clave-valor, pero difieren en lo siguiente:

**localStorage:** dura hasta que el código, el usuario o el navegador lo borren (sobrevive al cierre del navegador); su alcance es el origen (protocolo + dominio + puerto) y se comparte entre todas las pestañas y ventanas; capacidad aproximada de 5 MB; nunca se envía al servidor.

**sessionStorage:** dura lo que viva la pestaña (sobrevive a las recargas y se borra al cerrarla); su alcance es el origen, pero cada pestaña tiene su propia copia aislada; capacidad aproximada de 5 MB; nunca se envía al servidor.

**Cookies:** duran hasta el cierre de la sesión o hasta su fecha de expiración (Expires/Max-Age); su alcance es el dominio y la ruta, y se comparten entre pestañas; capacidad de unos 4 KB por cookie; se envían automáticamente al servidor en cada petición HTTP al dominio, lo que aumenta el tráfico.

**Qué elegiría.** Tema visual: localStorage, porque es una preferencia no sensible que debe durar entre visitas y no necesita viajar al servidor. Borrador de un formulario: sessionStorage, porque es un dato temporal ligado a la pestaña; no se mezcla con otras pestañas y desaparece al cerrarla (si debiera sobrevivir al cierre del navegador, se usaría localStorage). Identificador de una sesión autenticada: una cookie emitida por el servidor con los atributos HttpOnly, Secure y SameSite, porque JavaScript no puede leerla (reduce el robo por XSS), viaja sola al servidor, que es quien valida la sesión, y el servidor puede hacerla expirar o revocarla.

## 3. Web Storage y JSON

Web Storage solo almacena texto. Explica qué ocurre cuando se guarda un objeto directamente, por qué sucede y cómo JSON.stringify() y JSON.parse() resuelven el problema. Aclara además la diferencia entre un objeto de JavaScript y un texto JSON.

**Qué ocurre al guardar un objeto.** Web Storage solo almacena cadenas de texto. Si se pasa un objeto a setItem(), el navegador lo convierte implícitamente a texto con toString() y queda guardado como "[object Object]" (un arreglo quedaría como "a,b,c" y null como "null"). Sucede porque la especificación define el almacenamiento como un mapa de texto a texto que no conoce la estructura de los objetos. Al leerlo con getItem() solo se recupera esa cadena: las propiedades ya no existen (por ejemplo, estudiante.nombre es undefined) y la información original se perdió.

**Cómo lo resuelven JSON.stringify() y JSON.parse().** JSON.stringify(objeto) convierte el objeto en un texto JSON que sí describe su contenido, y ese texto se guarda sin pérdida. JSON.parse(texto) hace el camino inverso y reconstruye un objeto con sus propiedades y sus tipos (número, booleano, arreglo, null). Así, un número guardado vuelve a ser number y no la cadena "21".

**Objeto de JavaScript frente a texto JSON.** Un objeto de JavaScript es una estructura viva en memoria, con referencias, métodos y tipos como Date, Set, Map o undefined. Un texto JSON es solo una cadena con un formato de intercambio: claves entre comillas dobles y valores limitados a cadena, número, booleano, null, arreglo u objeto; no admite funciones, undefined, comentarios ni comillas simples. Por eso la serialización tiene límites: una Date se convierte en cadena ISO, un

Set queda como {} y las funciones y los valores undefined se omiten de los objetos.

## 4. Límites y programación defensiva

Analiza las limitaciones de Web Storage (cuota, disponibilidad, naturaleza síncrona y solo texto) y explica cómo la programación defensiva y la degradación elegante permiten que una aplicación siga funcionando cuando el almacenamiento falla.

**Limitaciones de Web Storage.** (1) Cuota: unos 5 MB por origen; al superarla, setItem() lanza QuotaExceededError. (2) Disponibilidad: puede estar bloqueado o no existir (el usuario bloquea los datos del sitio, ciertos modos privados, políticas corporativas, iframes de terceros); incluso acceder a window.localStorage puede lanzar SecurityError. (3) Naturaleza síncrona: cada lectura o escritura bloquea el hilo principal, de modo que guardar mucho o con mucha frecuencia degrada la interfaz; para grandes volúmenes conviene IndexedDB, que es asíncrona. (4) Solo texto: obliga a serializar y a validar los datos.

**Programación defensiva.** Parte de asumir que todo puede fallar: se comprueba la disponibilidad (escribir y borrar una clave de prueba dentro de un try/catch), se envuelven setItem(), getItem() y JSON.parse() en try/catch, se valida el tipo y la forma de lo leído (puede estar dañado o editado a mano), se usan valores por defecto y se versionan las claves.

**Degradación elegante.** Es el resultado de esa práctica: si el almacenamiento falla, la aplicación no se rompe; sigue funcionando con el estado en memoria, aunque no recuerde los datos entre visitas, y avisa al usuario. La persistencia se trata como una mejora y no como un requisito para que la funcionalidad principal opere.

## 5. Seguridad y alcance del navegador

Explica por qué el navegador es un entorno no confiable para datos críticos, qué información no debe guardarse en Web Storage y por qué el servidor conserva la autoridad sobre los datos. Aclara también por qué cachear datos de una API en localStorage no equivale a un proceso ETL.

**Un entorno no confiable.** El usuario controla por completo lo que hay en el navegador: con las herramientas de desarrollo cualquiera puede ver, editar o borrar lo guardado en Web Storage, y también puede leerlo otra persona en un equipo compartido, una extensión o un malware. Web Storage no está cifrado ni tiene un equivalente a HttpOnly: cualquier script que se ejecute en el origen (por ejemplo, tras un ataque XSS) puede leerlo. Los datos del cliente deben tratarse como una entrada externa: pueden faltar, estar dañados o haber sido manipulados.

**Qué no debe guardarse.** Contraseñas, tokens de acceso o de sesión (JWT), claves de API o secretos, datos personales o financieros sensibles (documentos de identidad, tarjetas, historiales) y cualquier información de la que dependan permisos, roles, precios o decisiones de seguridad.

**La autoridad del servidor.** El servidor es la fuente de verdad: valida y autoriza cada petición sin fiarse de lo que envíe el cliente. Lo almacenado en el navegador es solo una conveniencia (preferencias, borradores, caché) que puede perderse y que se reconstruye desde el servidor.

**Caché de una API frente a ETL.** ETL (Extraer, Transformar y Cargar) es un proceso de integración de datos que toma información de una o varias fuentes, la limpia y transforma y la carga en un almacén de destino confiable (como un data warehouse) para su análisis, con calidad y gobierno de datos. Cachear la respuesta de una API en localStorage es solo guardar una copia temporal para responder más rápido o funcionar sin conexión: vive en el equipo de un usuario, tiene vigencia (TTL), puede quedar desactualizada, se pierde al limpiar el navegador y no consolida ni garantiza la calidad de los datos.

# PARTE II. PRÁCTICA

*Ejercicios 6 a 20 · Progresivos, desarrollados en Visual Studio Code*

## 6. Preferencia de tema con localStorage

*Una página olvida el tema elegido por el usuario cada vez que se recarga.*

**Actividad:** Crea una página con un botón que alterne entre tema claro y oscuro. Guarda la elección en localStorage con una clave con prefijo y restáurala al cargar, validando que el valor leído sea uno de los dos permitidos.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|

script.js — taller-persistencia / ejercicio06

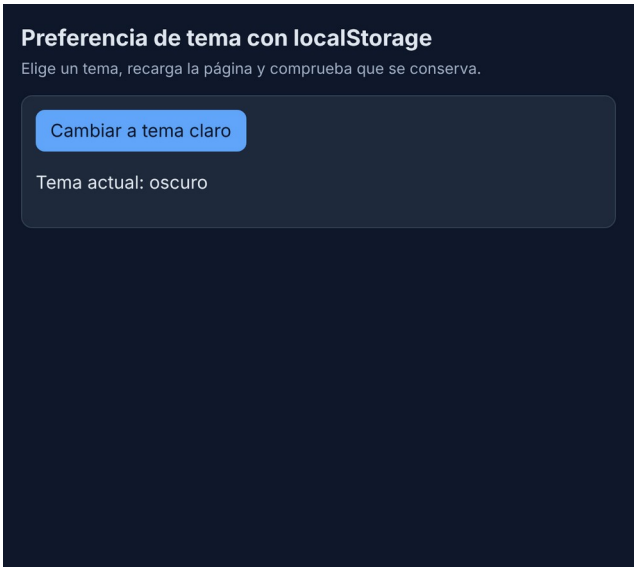
```
  try {
    const guardado = localStorage.getItem(CLAVE_TEMA);
    return TEMAS_VALIDOS.includes(guardado) ? guardado : "claro";
  } catch {
    return "claro"; // almacenamiento bloqueado: se usa el tema por defecto
  }
}

function guardarTema(tema) {
  try {
    localStorage.setItem(CLAVE_TEMA, tema);
  } catch {
    aviso.textContent = "No se pudo guardar el tema en este navegador.";
    aviso.hidden = false;
  }
}

function aplicarTema(tema) {
  document.documentElement.dataset.tema = tema;
  boton.textContent =
    tema === "oscuro" ? "Cambiar a tema claro" : "Cambiar a tema oscuro";
  boton.setAttribute("aria-pressed", String(tema === "oscuro"));
  estado.textContent = `Tema actual: ${tema}`;
}

let temaActual = leerTema();
aplicarTema(temaActual); // restaura al cargar

boton.addEventListener("click", () => {
  temaActual = temaActual === "claro" ? "oscuro" : "claro";
  aplicarTema(temaActual);
  guardarTema(temaActual);
});
```




**Enlace de GitHub (opcional):**

## 7. Contadores con localStorage y sessionStorage

*Se necesita observar de forma experimental la diferencia de alcance entre ambos mecanismos.*

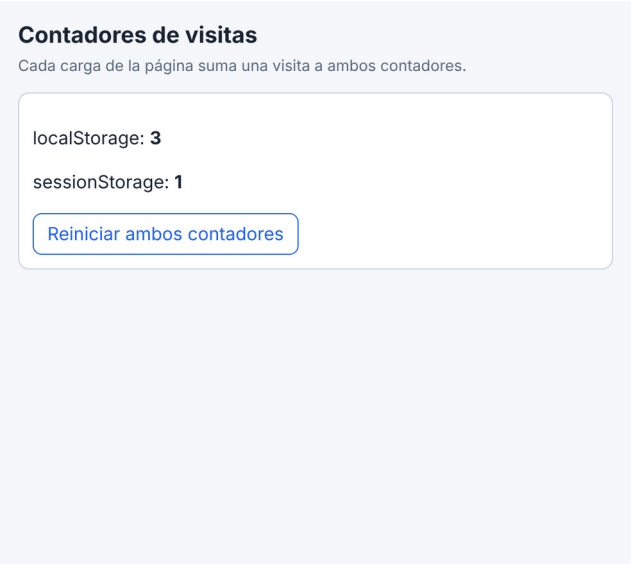
**Actividad:** Muestra dos contadores de visitas, uno en localStorage y otro en sessionStorage. Convierte los valores con Number() y String(), trata un dato ausente o no numérico como cero e incluye un botón que reinicie ambos con removeItem(). Abre la página en una segunda pestaña y anota en un comentario del código lo que observas.

**Evidencia 1: captura del código en Visual Studio Code**

script.js — taller-persistencia / ejercicio07

```
11
12 const spanLocal = document.getElementById("cont-local");
13 const spanSesion = document.getElementById("cont-sesion");
14 const botonReiniciar = document.getElementById("btn-reiniciar");
15
16 // Lo ausente (null) o no numérico (NaN) cuenta como cero.
17 function leerContador(almacen, clave) {
18   try {
19     const numero = Number(almacen.getItem(clave));
20     return Number.isFinite(numero) && numero >= 0 ? Math.floor(numero) : 0;
21   } catch {
22     return 0;
23   }
24 }
25
26 function escribirContador(almacen, clave, valor) {
27   try {
28     almacen.setItem(clave, String(valor));
29   } catch {
30     /* sin almacenamiento: el contador solo vive en pantalla */
31   }
32 }
33
34 function mostrar() {
35   spanLocal.textContent = String(leerContador(localStorage, CLAVE_LOCAL));
36   spanSesion.textContent = String(leerContador(sessionStorage, CLAVE_SESION));
37 }
38
39 function registrarVisita() {
40   escribirContador(localStorage, CLAVE_LOCAL,
41     leerContador(localStorage, CLAVE_LOCAL) + 1);
42   escribirContador(sessionStorage, CLAVE_SESION,
43     leerContador(sessionStorage, CLAVE_SESION) + 1);
```

**Evidencia 2: captura del resultado en el navegador**




**Enlace de GitHub (opcional):**

## 8. Recorrido de las entradas almacenadas

*Es necesario inspeccionar todo lo que una aplicación ha guardado.*

**Actividad:** Guarda tres entradas de ejemplo en localStorage y muestra en pantalla todas las claves y valores usando length y key(). Muestra también qué devuelve getItem() cuando la clave no existe.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|

script.js — taller-persistencia / ejercicio08

```
const total = document.getElementById("total");
const tabla = document.getElementById("tabla");
const inexistente = document.getElementById("inexistente");

function celda(texto) {
  const td = document.createElement("td");
  td.textContent = texto;
  return td;
}

try {
  // 1. Guardar las tres entradas de ejemplo.
  for (const [clave, valor] of Object.entries(ENTRADAS)) {
    localStorage.setItem(clave, valor);
  }

  // 2. Recorrer TODAS las entradas con length y key(i).
  total.textContent = String(localStorage.length);
  for (let i = 0; i < localStorage.length; i++) {
    const clave = localStorage.key(i);
    const fila = document.createElement("tr");
    fila.append(celda(String(i)), celda(clave), celda(localStorage.getItem(clave)));
    tabla.append(fila);
  }

  // 3. getItem con una clave que no existe devuelve null (no undefined).
  const valor = localStorage.getItem("pwc2:no-existe");
  inexistente.textContent = String(valor); // "null"
} catch (error) {
  total.textContent = "0";
  inexistente.textContent = `Almacenamiento no disponible (${error.name})`;
}
```

### Recorrido de las entradas almacenadas

Se guardan tres entradas y se recorren con length y key().

Total de entradas (`localStorage.length`): **3**

| # | Clave | Valor |
|---|---|---|
| 0 | pwc2:ej8:idioma | es-CO |
| 1 | pwc2:ej8:usuario | Ana Gómez |
| 2 | pwc2:ej8:ultima-visita | 2026-11-05 |

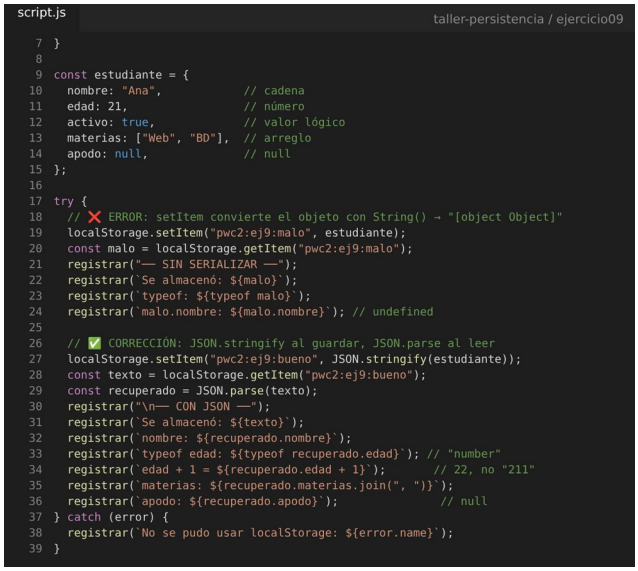
getItem("pwc2:no-existe") devuelve: `null`

**Enlace de GitHub (opcional):**

## 9. El error de guardar objetos sin serializar

*Un estudiante guarda un objeto en localStorage y, al recuperarlo, sus datos no aparecen.*

**Actividad:** Reproduce el error con un objeto que tenga cadena, número, valor lógico, arreglo y null, y muestra en consola qué se almacenó realmente. Luego corrígelo con JSON.stringify() y JSON.parse(), y comprueba que el número recuperado sigue siendo numérico.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|
|  |  |



**Enlace de GitHub (opcional):**

## 10. Comprobación de disponibilidad

*El almacenamiento puede estar bloqueado o no estar disponible.*

**Actividad:** Implementa la función almacenamientoDisponible(tipo) con try/catch, que devuelva true o false para localStorage y sessionStorage. Muestra en la página un mensaje distinto según el resultado.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|
|  |  |





**Enlace de GitHub (opcional):**

## 11. Lista de tareas persistente

*La lista de tareas del código base pierde todo al recargar la página.*

**Actividad:** Modifica el código base para que cada tarea sea un objeto con id, texto y hecha. Guarda el arreglo completo en localStorage como JSON, con una clave con prefijo y versión, cada vez que cambie, y restáuralo al cargar la página.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|

script.js — taller-persistencia / ejercicio11

```
// Ejercicio 11 — Lista de tareas persistente
// Cada tarea es un objeto { id, texto, hecha }. Clave con prefijo y versión.
const CLAVE_TAREAS = "pwc2:tareas:v1";

const form = document.getElementById("form-tarea");
const campo = document.getElementById("texto-tarea");
const lista = document.getElementById("lista");
const aviso = document.getElementById("aviso");

function cargarTareas() {
  try {
    const datos = JSON.parse(localStorage.getItem(CLAVE_TAREAS));
    return Array.isArray(datos) ? datos : [];
  } catch {
    return [];
  }
}

// Se guarda el ARREGLO COMPLETO cada vez que algo cambia.
function guardarTareas() {
  try {
    localStorage.setItem(CLAVE_TAREAS, JSON.stringify(tareas));
    aviso.hidden = true;
  } catch {
    aviso.textContent = "No se pudo guardar: los cambios se perderán al recargar.";
    aviso.hidden = false;
  }
}

let tareas = cargarTareas(); // restaura al cargar

function siguienteId() {
  return tareas.reduce((maximo, t) => Math.max(maximo, t.id), 0) + 1;
```




**Enlace de GitHub (opcional):**

## 12. Validación de los datos leídos

*Los datos guardados pueden estar dañados o haber sido editados por el usuario.*

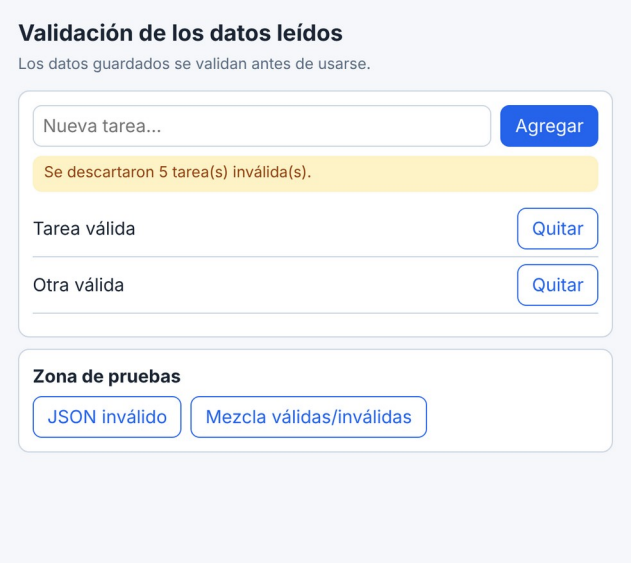
**Actividad:** Agrega una función que valide cada tarea leída (tipos de id, texto y hecha), descarte las inválidas y garantice que el resultado sea un arreglo. Protege JSON.parse() con try/catch y usa una lista vacía como valor por defecto. Prueba editando el valor a mano en las herramientas de desarrollo con un JSON inválido.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|

script.js — taller-persistencia / ejercicio12

```
// Valida los tipos de id, texto y hecha de UNA tarea.
function esTareaValida(t) {
  return (
    typeof t === "object" && t !== null && !Array.isArray(t) &&
    Number.isInteger(t.id) && t.id > 0 &&
    typeof t.texto === "string" && t.texto.trim() !== "" &&
    typeof t.hecha === "boolean"
  );
}

// Siempre devuelve { tareas: Array, descartadas: número, ilegible: boolean }.
function leerTareas() {
  const resultado = { tareas: [], descartadas: 0, ilegible: false };
  let datos;
  try {
    const texto = localStorage.getItem(CLAVE_TAREAS);
    if (texto === null) return resultado;  // primera visita: lista vacía
    datos = JSON.parse(texto);             // puede lanzar SyntaxError
  } catch {
    resultado.ilegible = true;             // valor por defecto: lista vacía
    return resultado;
  }
  if (!Array.isArray(datos)) {             // JSON válido pero no es arreglo
    resultado.ilegible = true;
    return resultado;
  }
  const idsVistos = new Set();
  for (const t of datos) {
    if (esTareaValida(t) && !idsVistos.has(t.id)) {
      idsVistos.add(t.id);
      resultado.tareas.push({ id: t.id, texto: t.texto, hecha: t.hecha });
    } else {
      resultado.descartadas++;             // inválida o id repetido
```




**Enlace de GitHub (opcional):**

## 13. Tareas completadas y eliminación

*La lista necesita reflejar el estado de cada tarea y mantenerse ordenada.*

**Actividad:** Agrega una casilla para marcar cada tarea como hecha (con el texto tachado) y un botón «Eliminar completadas». Persiste cada cambio y muestra un resumen de tareas pendientes en una región aria-live. Construye los elementos con createElement() y textContent, sin usar innerHTML.

**Evidencia 1: captura del código en Visual Studio Code**

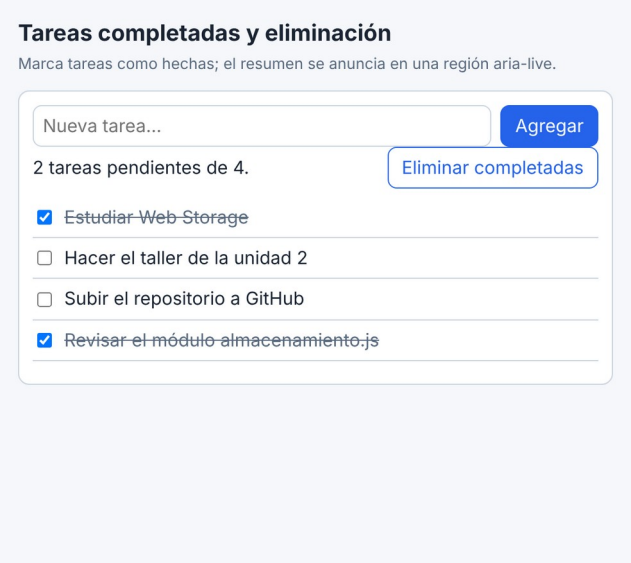
script.js — taller-persistencia / ejercicio13

```
function esTareaValida(t) {
  return (
    typeof t === "object" && t !== null && !Array.isArray(t) &&
    Number.isInteger(t.id) && t.id > 0 &&
    typeof t.texto === "string" && t.texto.trim() !== "" &&
    typeof t.hecha === "boolean"
  );
}

function leerTareas() {
  try {
    const datos = JSON.parse(localStorage.getItem(CLAVE_TAREAS));
    if (!Array.isArray(datos)) return [];
    const ids = new Set();
    return datos.filter((t) => {
      if (!esTareaValida(t) || ids.has(t.id)) return false;
      ids.add(t.id);
      return true;
    });
  } catch {
    return [];
  }
}

function guardarTareas() {
  try {
    localStorage.setItem(CLAVE_TAREAS, JSON.stringify(tareas));
    aviso.hidden = true;
  } catch {
    aviso.textContent = "No se pudo guardar: los cambios se perderán al recargar.";
    aviso.hidden = false;
  }
}
```

**Evidencia 2: captura del resultado en el navegador**




**Enlace de GitHub (opcional):**

## 14. Tipos de datos que no sobreviven a la serialización

*Una tarea con fecha de vencimiento y etiquetas pierde información al recuperarse.*

**Actividad:** Crea una tarea con una fecha (Date) y un conjunto de etiquetas (Set). Muestra en pantalla qué ocurre con cada uno al serializar sin tratamiento. Luego convierte el Set en arreglo antes de guardar, usa un reviver para recuperar la fecha como Date y verifica el resultado con instanceof.

**Evidencia 1: captura del código en Visual Studio Code**

script.js — taller-persistencia / ejercicio14

```
};

try {
  // 1. SIN TRATAMIENTO: se pierde información
  const sinTratar = JSON.stringify(tarea);
  const leidaMal = JSON.parse(sinTratar);
  registrar("— SIN TRATAMIENTO —");
  registrar(`JSON: ${sinTratar}`);
  registrar(`vence → ${typeof leidaMal.vence} (ya no es Date)`);
  registrar(`etiquetas → ${JSON.stringify(leidaMal.etiquetas)} (el Set quedó vacío)`);

  // 2. CON TRATAMIENTO: Set → arreglo al guardar, reviver para la fecha
  const paraGuardar = { ...tarea, etiquetas: [...tarea.etiquetas] };
  localStorage.setItem(CLAVE, JSON.stringify(paraGuardar));

  const reviver = (clave, valor) => {
    if (clave === "vence" && typeof valor === "string") {
      const fecha = new Date(valor);
      return Number.isNaN(fecha.getTime()) ? null : fecha;
    }
    return valor;
  };
  const recuperada = JSON.parse(localStorage.getItem(CLAVE), reviver);
  recuperada.etiquetas = new Set(recuperada.etiquetas); // arreglo → Set

  registrar("\n— CON TRATAMIENTO —");
  registrar(`Guardado: ${localStorage.getItem(CLAVE)}`);
  registrar(`vence instanceof Date → ${recuperada.vence instanceof Date}`);
  registrar(`vence (ISO) → ${recuperada.vence.toISOString()}`);
  registrar(`etiquetas instanceof Set → ${recuperada.etiquetas instanceof Set}`);
  registrar(`etiquetas → ${[...recuperada.etiquetas].join(", ")}`);
} catch (error) {
  registrar(`No se pudo usar localStorage: ${error.name}`);
```

**Evidencia 2: captura del resultado en el navegador**

**Tipos que no sobreviven a la serialización**

Date y Set antes y después del tratamiento.

```
— SIN TRATAMIENTO —
JSON: {"id":1,"texto":"Entregar el taller","vence":"2026-11-
20T12:00:00.000Z","etiquetas":{}}
vence → string (ya no es Date)
etiquetas → {} (el Set quedó vacío)

— CON TRATAMIENTO —
Guardado: {"id":1,"texto":"Entregar el taller","vence":"2026-11-
20T12:00:00.000Z","etiquetas":["web","taller"]}
vence instanceof Date → true
vence (ISO) → 2026-11-20T12:00:00.000Z
etiquetas instanceof Set → true
etiquetas → web, taller
```

**Enlace de GitHub (opcional):**

## 15. Exclusión de campos con replacer

*Un objeto de usuario contiene un token que no debe terminar en el almacenamiento.*

**Actividad:** Crea un objeto usuario con id, nombre y token. Serialízalo excluyendo el token de dos maneras (lista blanca de claves y función replacer), guarda en localStorage la versión filtrada y muestra el resultado en pantalla.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|

script.js — taller-persistencia / ejercicio15

```
// Ejercicio 15 — Exclusión de campos con replacer
const CLAVE = "pwc2:ej15:usuario";
const salida = document.getElementById("salida");

const usuario = { id: 7, nombre: "Laura Gómez", token: "abc123-secreto" };

// Opción A: LISTA BLANCA de claves (solo se serializan las listadas)
const conListaBlanca = JSON.stringify(usuario, ["id", "nombre"]);

// Opción B: FUNCIÓN REPLACER (devolver undefined omite la propiedad)
const conReplacer = JSON.stringify(usuario, (clave, valor) =>
  clave === "token" ? undefined : valor
);

const lineas = [
  `Objeto original: ${JSON.stringify(usuario)}`,
  `A) Lista blanca:  ${conListaBlanca}`,
  `B) Replacer:      ${conReplacer}`,
  `¿A y B son iguales? ${conListaBlanca === conReplacer}`,
];

try {
  localStorage.setItem(CLAVE, conReplacer); // se guarda la versión filtrada
  const guardado = localStorage.getItem(CLAVE);
  lineas.push(
    "",
    `Guardado en localStorage: ${guardado}`,
    `¿Contiene "token"? ${guardado.includes("token")}`
  );
} catch (error) {
  lineas.push("", `No se pudo guardar: ${error.name}`);
}

```

**Exclusión de campos con replacer**

El token nunca debe llegar al almacenamiento.

```
Objeto original: {"id":7,"nombre":"Laura Gómez","token":"abc123-
secreto"}
A) Lista blanca:  {"id":7,"nombre":"Laura Gómez"}
B) Replacer:      {"id":7,"nombre":"Laura Gómez"}
¿A y B son iguales? true

Guardado en localStorage: {"id":7,"nombre":"Laura Gómez"}
¿Contiene "token"? false
```

**Enlace de GitHub (opcional):**

## 16. Módulo reutilizable de almacenamiento

*Repetir try/catch, stringify y parse en cada punto del programa genera código duplicado.*

**Actividad:** Crea un archivo almacenamiento.js que exporte almacenamientoDisponible, guardarJSON, leerJSON y eliminar, recibiendo el tipo "local" o "session". Ninguna función debe lanzar excepciones hacia la aplicación. Importa el módulo con type="module" y refactoriza la lista de tareas para usarlo.

**Evidencia 1: captura del código en Visual Studio Code**

almacenamiento.js — taller-persistencia

```
// almacenamiento.js — Módulo reutilizable para Web Storage (ejercicio 16).
// Regla de diseño: NINGUNA función lanza excepciones hacia la aplicación.
// Si algo falla, devuelven false / el valor por defecto.

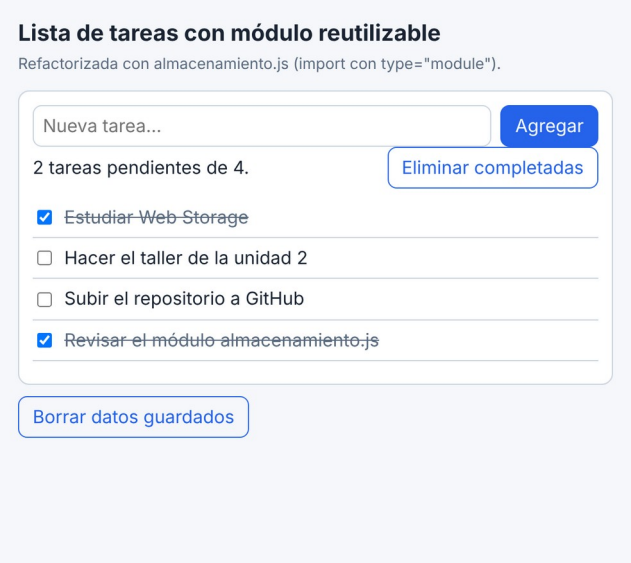
const TIPOS = { local: "localStorage", session: "sessionStorage" };

// Devuelve el objeto Storage o null. Incluso acceder a window.localStorage
// puede lanzar SecurityError (datos del sitio bloqueados), por eso va en try.
function obtenerAlmacen(tipo) {
  try {
    if (!Object.hasOwn(TIPOS, tipo)) return null;
    return window[TIPOS[tipo]] ?? null;
  } catch {
    return null;
  }
}

export function almacenamientoDisponible(tipo) {
  try {
    const almacen = obtenerAlmacen(tipo);
    if (!almacen) return false;
    const prueba = "__prueba_almacenamiento__";
    almacen.setItem(prueba, "1");
    almacen.removeItem(prueba);
    return true;
  } catch {
    return false;
  }
}

export function guardarJSON(tipo, clave, valor, replacer) {
  try {
    const almacen = obtenerAlmacen(tipo);
```

**Evidencia 2: captura del resultado en el navegador**




**Enlace de GitHub (opcional):**

## 17. Borrador de formulario con sessionStorage

*Un usuario pierde lo escrito en un formulario de contacto al recargar accidentalmente la página.*

**Actividad:** Crea un formulario con nombre, correo y mensaje que guarde su borrador como JSON en sessionStorage en cada evento input y lo restaure al cargar con un aviso. Elimina el borrador al enviar con datos válidos (usa la validación nativa del navegador) y comprueba que no aparece en una pestaña nueva.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|

main.js — taller-persistencia / ejercicio17

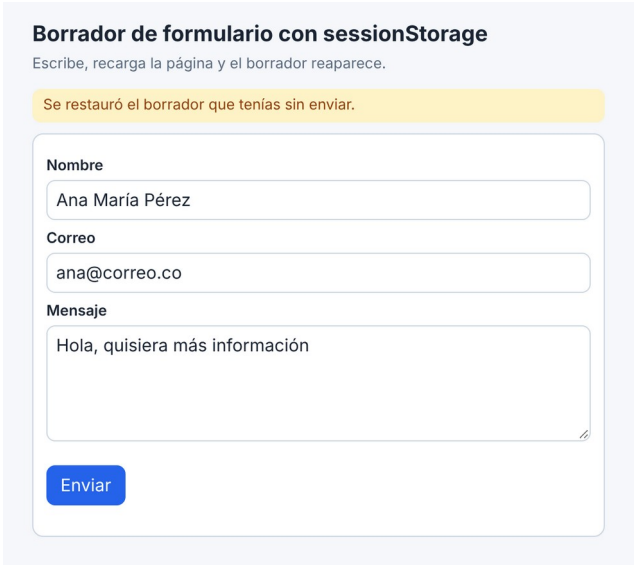
```
}

// Guarda el borrador en CADA evento input (la delegación cubre los 3 campos).
form.addEventListener("input", () => {
  const borrador = leerFormulario();
  if (CAMPOS.every((campo) => borrador[campo] === "")) {
    eliminar("session", CLAVE_BORRADOR);   // formulario vacío: sin borrador
  } else {
    guardarJSON("session", CLAVE_BORRADOR, borrador);
  }
});

// Restaura al cargar, validando la forma del borrador.
function restaurar() {
  const borrador = leerJSON("session", CLAVE_BORRADOR, null);
  const valido =
    borrador !== null && typeof borrador === "object" &&
    CAMPOS.every((campo) => typeof borrador[campo] === "string");
  if (!valido) return;
  for (const campo of CAMPOS) form.elements[campo].value = borrador[campo];
  mostrarAviso("Se restauró el borrador que tenías sin enviar.");
}

// El evento "submit" solo ocurre si la validación nativa del navegador
// (required, type="email", minlength) aprobó los datos.
form.addEventListener("submit", (evento) => {
  evento.preventDefault();
  eliminar("session", CLAVE_BORRADOR);     // borrador ya no necesario
  form.reset();
  mostrarAviso("✅ Mensaje enviado. El borrador fue eliminado.");
});

restaurar();
```




**Enlace de GitHub (opcional):**

## 18. Sincronización entre pestañas con el evento storage

*Dos pestañas abiertas de la lista de tareas muestran información distinta.*

**Actividad:** Agrega a la lista de tareas un manejador del evento storage que, solo cuando cambie la clave de las tareas, vuelva a leer y dibujar la lista. Comprueba el funcionamiento con dos pestañas y explica en un comentario por qué el evento no se dispara en la pestaña que realizó el cambio.

**Evidencia 1: captura del código en Visual Studio Code**

main.js — taller-persistencia / ejercicio18

```
btnBorrarTodo.textContent = "Borrar datos guardados";
btnBorrarTodo.addEventListener("click", () => {
  eliminar("local", CLAVE_TAREAS);
  tareas = [];
  dibujar();
});
document.querySelector("main").append(btnBorrarTodo);

// ── Ejercicio 18: sincronización entre pestañas ──────────────────
// Por especificación, el evento "storage" se envía solo a los OTROS
// documentos (pestañas/ventanas) del mismo origen que comparten el área de
// almacenamiento; NUNCA a la pestaña que hizo el cambio. Esa pestaña ya
// conoce el nuevo estado (lo tiene en memoria y ya redibujó la lista), por
// eso no necesita enterarse: el evento existe para avisar a las demás.
window.addEventListener("storage", (evento) => {
  if (evento.storageArea !== localStorage) return;  // ignora sessionStorage
  // key === null significa que otra pestaña ejecutó localStorage.clear()
  if (evento.key !== CLAVE_TAREAS && evento.key !== null) return;
  tareas = leerTareas();                            // vuelve a leer
  dibujar();                                        // vuelve a dibujar
  mostrarAviso("Lista actualizada desde otra pestaña.");
});

dibujar();
```

**Evidencia 2: captura del resultado en el navegador**




**Enlace de GitHub (opcional):**

## 19. Versionado y migración de datos

*Una nueva versión de la aplicación cambia el formato de los datos guardados por la anterior.*

**Actividad:** Implementa la función migrar(datos) que convierta un perfil de la versión 1 (nombre completo en un solo campo) a la versión 2 (nombre y apellido por separado), devuelva sin cambios los datos de la versión actual y descarte con null los valores que no sean objetos o tengan una versión desconocida. Guarda el resultado en localStorage y muéstralo en pantalla.

**Evidencia 1: captura del código en Visual Studio Code**

main.js — taller-persistencia / ejercicio19

```
}

export function migrar(datos) {
  // 1. Solo se aceptan objetos (no null, no arreglos, no primitivos).
  if (typeof datos !== "object" || datos === null || Array.isArray(datos)) {
    return null;
  }
  // 2. Versión actual: se devuelve SIN cambios.
  if (datos.version === VERSION_ACTUAL) return datos;
  // 3. Versión 1 → 2
  if (datos.version === 1) {
    if (typeof datos.nombreCompleto !== "string" ||
        datos.nombreCompleto.trim() === "") {
      return null; // v1 sin nombre utilizable: se descarta
    }
    const { nombreCompleto, ...resto } = datos; // conserva otros campos
    return { ...resto, version: 2, ...separarNombre(nombreCompleto) };
  }
  // 4. Versión desconocida (0, 3, "2", undefined...): se descarta
  return null;
}

const salida = document.getElementById("salida");
const tablaCasos = document.getElementById("casos");

function mostrarFila(entrada, resultado) {
  const fila = document.createElement("tr");
  for (const valor of [entrada, resultado]) {
    const celda = document.createElement("td");
    celda.textContent = JSON.stringify(valor);
    fila.append(celda);
  }
  tablaCasos.append(fila);
```

**Evidencia 2: captura del resultado en el navegador**

### Versionado y migración de datos

Un perfil v1 guardado por la versión anterior pasa a v2.

```
Guardado antes:   {"version":1,"nombreCompleto":"Ana María Pérez Gómez","ciudad":"Manizales"}
Resultado migrar: {"version":2,"ciudad":"Manizales","nombre":"Ana María","apellido":"Pérez Gómez"}
Guardado ahora:   {"version":2,"ciudad":"Manizales","nombre":"Ana María","apellido":"Pérez Gómez"}
```

**Casos de prueba de migrar(datos)**

| Entrada | Resultado |
|---|---|
| {"version":1,"nombreCompleto":"Juan Pérez"} | {"version":2,"nombre":"Juan","apellido":"Pérez"} |
| {"version":2,"nombre":"Laura","apellido":"Gómez"} | {"version":2,"nombre":"Laura","apellido":"Gómez"} |
| {"version":99,"nombre":"X"} | null |
| "texto" | null |
| 42 | null |
| null | null |
| [1,2] | null |
| {"sinVersion":true} | null |

**Enlace de GitHub (opcional):**

## 20. Caso integrador: directorio de usuarios con caché

*Un directorio de usuarios debe responder con rapidez en visitas repetidas y seguir siendo útil sin conexión.*

**Actividad:** Consume la API jsonplaceholder.typicode.com/users con fetch (verificando respuesta.ok) y transforma los datos conservando solo id, nombre, correo y empresa. Guarda la respuesta en localStorage con su marca de tiempo y una vigencia (TTL) de 10 minutos, usando la caché vencida si la red falla. Guarda los favoritos en localStorage (convirtiendo el Set en arreglo) y el texto del buscador en sessionStorage. Muestra un mensaje que indique el origen de los datos. Utiliza el módulo del ejercicio 16.

| Evidencia 1: captura del código en Visual Studio Code | Evidencia 2: captura del resultado en el navegador |
|---|---|

main.js — taller-persistencia / ejercicio20

```
async function obtenerUsuarios({ forzar = false } = {}) {
  const cache = leerCache();
  const edad = cache ? Date.now() - cache.guardadoEn : Infinity;
  const vigente = cache !== null && edad >= 0 && edad < TTL_MS;

  if (vigente && !forzar) {
    return { datos: cache.datos, origen: "cache", guardadoEn: cache.guardadoEn };
  }

  const control = new AbortController();
  const temporizador = setTimeout(() => control.abort(), 8000);
  try {
    const respuesta = await fetch(URL_API, { signal: control.signal });
    if (!respuesta.ok) throw new Error(`HTTP ${respuesta.status}`);
    const datos = transformar(await respuesta.json());
    if (datos.length === 0) throw new Error("Respuesta sin usuarios válidos");
    guardarCache(datos);
    return { datos, origen: "red", guardadoEn: Date.now() };
  } catch {
    if (cache) { // la red falló: se usa la caché aunque esté vencida
      return {
        datos: cache.datos,
        origen: vigente ? "cache" : "cache-vencida",
        guardadoEn: cache.guardadoEn,
      };
    }
    return { datos: [], origen: "error", guardadoEn: null };
  } finally {
    clearTimeout(temporizador);
  }
}

// ── Favoritos (Set ⇄ arreglo) y buscador (sessionStorage) ──────────────
```




**Enlace de GitHub (opcional):**

## ENTREGA FINAL

Pega el enlace completo del repositorio de GitHub donde se encuentran todos los ejercicios.

**Enlace del repositorio:**